

GenAI. Lecture 2 - GAN

A. Ryzhikov¹, S. Ali¹, A. Akhmatova¹, T. Ramazyan¹, K. Aithadzhaev¹, A. Khizhik¹

¹National Research University Higher School of Economics



January 20, 2025



LAMBDA • HSE

Recap



Recap. Key points

- ▶ Our course is how to sample $x \sim P(x)$
- ▶ Multidimensional challenge
- ▶ One way to learn is distribution matching
- ▶ $D(P_{\text{real}}, Q)$ is objective
- ▶ $D(P_{\text{real}}, Q)$ can be approximated (lower-bound) by auxiliary neural network T
- ▶ *Fidelity* (FID) and *diversity* (IS) metrics are analogous of precision and recall

Recap. f-divergence

$$D_f(p, q) = \int q(x) f\left(\frac{p(x)}{q(x)}\right) dx$$

$$f^*(t) = \sup_{u \in \text{dom}_f} \{ut - f(u)\}$$

$$D_f(P \| Q) = \int_x q(x) \sup_{t \in \text{dom}_{f^*}} \left\{ t \frac{p(x)}{q(x)} - f^*(t) \right\} dx$$

$$\geq \sup_{T \in \mathcal{T}} \left(\int_x p(x) T(x) dx - \int_x q(x) f^*(T(x)) dx \right)$$

$$= \sup_{T \in \mathcal{T}} (\mathbb{E}_{x \sim P}[T(x)] - \mathbb{E}_{x \sim Q}[f^*(T(x))]) ,$$

GAN



(f-)GAN

$$D_f(P\|Q) \geq \sup_{T \in \mathcal{T}} (\mathbb{E}_{x \sim P}[T(x)] - \mathbb{E}_{x \sim Q}[f^*(T(x))]) = L$$

Let $T \approx T_\eta$ (neural network with learnable parameters η)

$$\min_{\theta} \max_{\eta} \mathbb{E}_{x \sim P}[T_\eta(x)] - \mathbb{E}_{x \sim Q}[f^*(T_\eta(x))]$$

Q_θ - generator, T_η - discriminator

The generator minimizes D w.t. θ , whereas D is estimated by auxiliary neural network T (discriminator).

rKL-divergence. Vanilla GAN.

Let

$$D(P_{\text{real}}, Q_{\theta}) = KL(Q\|P) = \int_{x \in \Omega} Q(x) \log \left(\frac{Q(x)}{P(x)} \right) dx$$

Expanding the KL divergence:

$$KL(Q\|P) = \int_{x \in \Omega} Q(x) \log(Q(x)) dx - \int_{x \in \Omega} Q(x) \log(P(x)) dx$$

Which can be rewritten as:

$$KL(Q\|P) = \text{Entropy}(Q) - \int_{x \in \Omega} Q(x) \log(P(x)) dx \approx \Sigma_{x \sim Q} \log D(x)$$

$$\min_{\theta} KL(Q_{\theta}\|P) = \min_{\theta} \Sigma_{x \sim Q} \log D(x) = \min_{\theta} \Sigma_{x \sim Q} \log D(x) + \min_{\theta} \Sigma_{x \sim P} \log(1 - D(x))$$

Discriminator Optimization in GANs

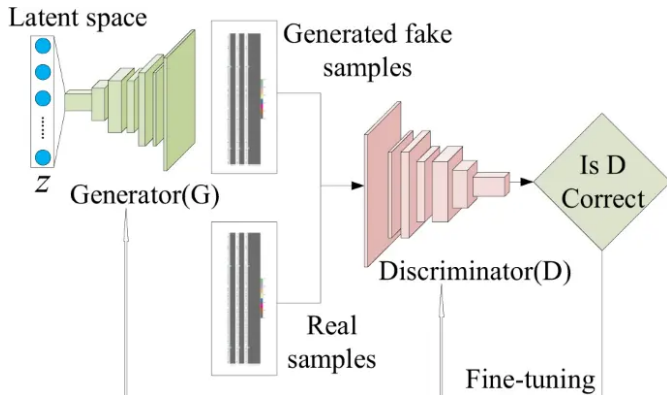
- ▶ Add a classifying neural network, **discriminator** D_ϕ , to distinguish between the real and generated samples.
- ▶ Optimize:

$$\max_{\phi} \left(\mathbb{E}_{x \sim p(x)} \log(D_\phi(x)) + \mathbb{E}_{x \sim q_\theta(x)} \log(1 - D_\phi(x)) \right)$$

Real samples

Generated samples

Vanilla GAN



$$\min_{\theta} KL(Q_{\theta} \| P) = \min_{\theta} \sum_{x \sim Q} \log D(x) + \min_{\theta} \sum_{x \sim P} \log(1 - D(x))$$

Here, $D(x) \approx p(x)$ (p.d.f.)

Binary Cross-Entropy (BCE) Loss:

$$\text{BCE}(D_\phi) = -\mathbb{E}_{x \sim P(x)} \log(D_\phi(x)) - \mathbb{E}_{x \sim Q_\theta(x)} \log(1 - D_\phi(x))$$

Optimal Discriminator:

$$D_\phi^*(x) = \frac{P(x)}{P(x) + Q_\theta(x)}$$

Substitute $D_\phi^*(x)$ into BCE:

$$\text{BCE}(D_\phi^*) = -\mathbb{E}_{x \sim P(x)} \log\left(\frac{P(x)}{P(x) + Q_\theta(x)}\right) - \mathbb{E}_{x \sim Q_\theta(x)} \log\left(\frac{Q_\theta(x)}{P(x) + Q_\theta(x)}\right)$$

Simplified Relation:

$$\text{BCE}(D_\phi^*) = -\log(4) + 2 \cdot \text{JS}(P \| Q_\theta)$$

GAN enhancements



LapGAN



Figure: Laplacian GAN

Progressive Growing

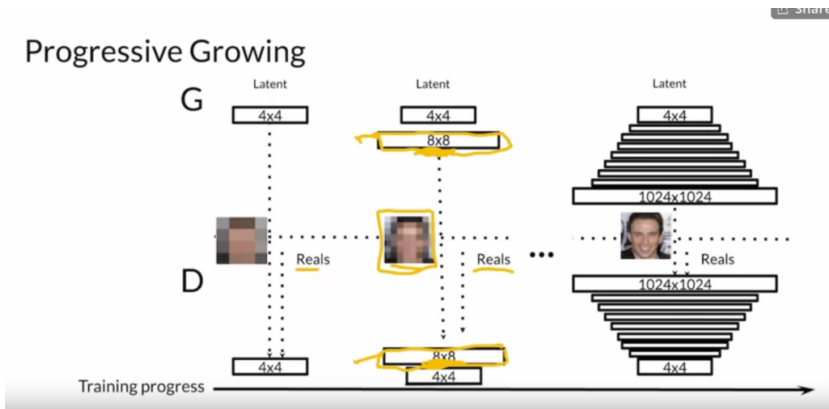


Figure: Progressive growing GAN

CGAN

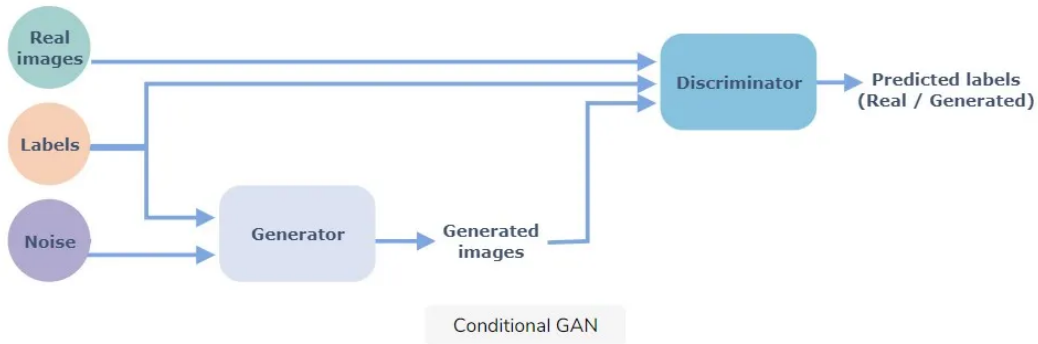


Figure: CGAN (conditional)

InfoGAN and SCGAN

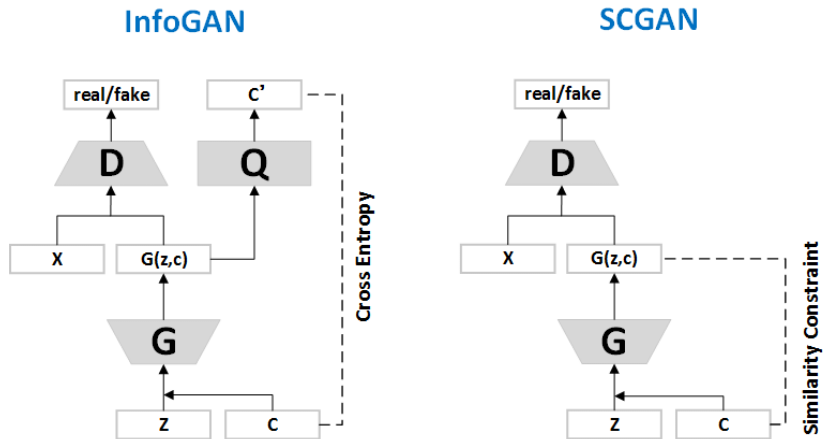


Figure: InfoGAN vs. SCGAN

SAGAN

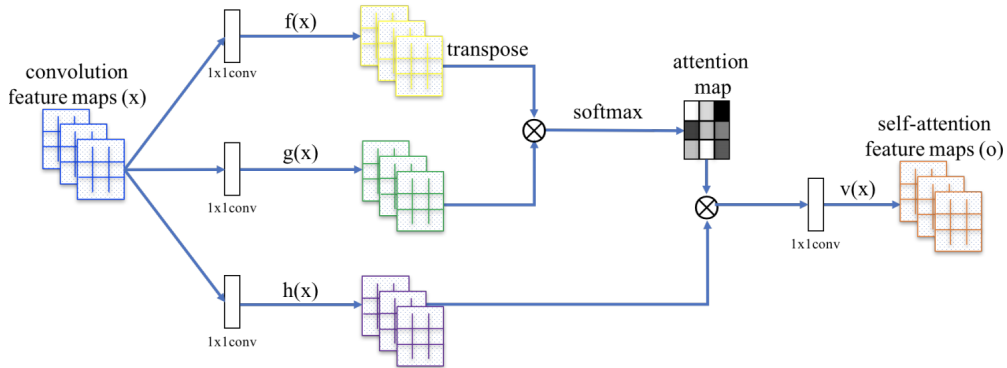


Figure: SAGAN

StyleGAN

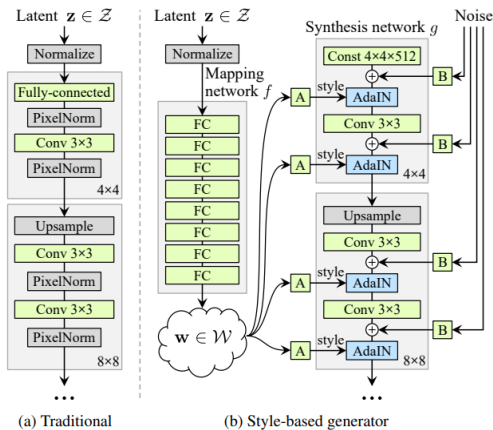


Figure: StyleGAN

GAN tricks

- ▶ **Use Label Smoothing:** For real labels, use 0.9 instead of 1.0. (Prevents the discriminator from becoming overconfident.)
- ▶ **Add Noisy Labels:** Randomly flip labels (e.g., 5% of real labels become fake). (Helps prevent overfitting.)
- ▶ **Normalize Inputs:** Ensure inputs to the discriminator and generator are normalized to $[-1, 1]$.
- ▶ **Use Batch Normalization:** Apply batch normalization in both generator and discriminator networks. (Helps stabilize gradients.)
- ▶ **Architectural Choices:** Use Leaky ReLU in the discriminator. Avoid using max pooling or fully connected layers.
- ▶ **Balance Training:** Train discriminator and generator alternately. Avoid overtraining the discriminator.

<https://github.com/soumith/ganhacks>

GAN tricks

- ▶ **Use the Log Trick:** For the generator, minimize $-\log(D(G(z)))$ instead of maximizing $\log(1 - D(G(z)))$. (Helps avoid vanishing gradients.)
- ▶ **Add Noise:** Add noise to discriminator inputs or weights. (Helps prevent discriminator overfitting.)
- ▶ **Spectral Normalization:** Apply spectral normalization to the discriminator to stabilize training.
- ▶ **Gradient Penalty:** Add a gradient penalty term (e.g., in WGAN-GP) to enforce Lipschitz continuity.
- ▶ **Two-Time-Scale Update Rule (TTUR):** Use different learning rates for generator and discriminator (e.g., 10^{-4} for generator, 4×10^{-4} for discriminator).

<https://github.com/soumith/ganhacks>

GAN tricks. Hyperparameters

- ▶ **Use Adam Optimizer for Discriminator and SGD for generator:** The discriminator often converges faster and benefits from adaptive optimizers like Adam. The generator may benefit from SGD to stabilize its slower learning and prevent overfitting to discriminator feedback.
- ▶ **Batch Size:** Start with a batch size of 64 or 128. Larger batches can improve stability but require more memory.
- ▶ **Begin with a simple architecture** like DCGAN before experimenting with more complex models.

<https://github.com/soumith/ganhacks>

GAN issues



GAN issues

- ▶ vanishing gradients
- ▶ mode collapse
- ▶ bad convergence
- ▶ minimax optimization lead to instability in training
- ▶ sensitive to hyperparameters and architecture choice